

Exercice 1 :

```
#!/bin/sh
# Get machine name and display user info using environment variables
machine=$(uname)
echo "Vous êtes l'utilisateur $USER sur la machine $machine, votre répertoire utilisateur est $HOME"
```

Exercice 2 :

```
#!/bin/sh
# Check if argument is provided
if [ $# -le 0 ]
then
    echo "Il manque un argument"
else
    # Try to create directory
    mkdir $1
    # $? contains exit status of last command (0 = success)
    if [ $? -eq 0 ]
    then
        echo "le dossier $1 à été bien créé"
    else
        echo "le dossier $1 existe déjà"
    fi
fi
```

Exercice 3 :

```
#!/bin/sh
# Number guessing game

# Get secret number from first player
echo "Veuillez entrer une valeur à deviner entre 1 et 100"
read a
clear

# Get guess from second player
echo "Veuillez entrer un nombre entre (1 - 100)"
read x

# Loop until correct guess
while [ $x -ne $a ]; do
    clear
    # Give hint based on comparison
    if [ $x -gt $a ]; then
        echo "plus petit"
    else
        echo "plus grand"
    fi
    echo "Veuillez entrer un nombre entre (1 - 100)"
    read x
done

echo "Bravo !"
```

Exercice 4 :

```
#!/bin/sh
# Display statistics about a directory: files, subdirectories, total size

# Error function: writes the message on stderr and exits with code 1
erreur() {
    echo "Erreur : $1" >&2
    exit 1
}

# Check that exactly one parameter was given
if [ $# -ne 1 ]; then
    erreur "usage : $0 <repertoire>"
fi

# Check that this parameter is really a directory
if [ ! -d "$1" ]; then
    erreur "le repertoire \"$1\" n'existe pas"
fi

# Counters
nb_fichiers=0
nb_repertoires=0
taille_totale=0

# Loop over every entry of the directory
for element in "$1"/*; do
    # When the directory is empty the pattern is not expanded: skip it
    [ -e "$element" ] || continue

    if [ -d "$element" ]; then
        nb_repertoires=$((nb_repertoires + 1))
    elif [ -f "$element" ]; then
        nb_fichiers=$((nb_fichiers + 1))
        taille=$(wc -c < "$element")
        taille_totale=$((taille_totale + taille))
    fi
done

echo "Fichiers ordinaires : $nb_fichiers"
echo "Sous-répertoires   : $nb_repertoires"
echo "Taille totale      : $taille_totale octets"
```

Points à retenir :

- La **fonction** `erreur` est déclarée avant d'être appelée. Elle centralise les deux cas d'erreur : on écrit le message une seule fois au lieu de le répéter.
- `&2` redirige vers la **sortie d'erreur**. Un message d'erreur ne doit jamais partir sur la sortie standard, sinon il se retrouve mélangé au résultat si on redirige le script dans un fichier.
- `exit 1` renvoie un **code de retour** non nul : le script signale son échec au shell appelant, qui peut le tester avec `$?` (cf. exercice 2).
- `=$(( ... ))` fait du calcul entier. `compteur=$((compteur + 1))` incrémente le compteur.
- `$(wc -c < "$fichier")` capture la sortie de la commande dans une variable. On utilise `<` plutôt que `wc -c "$fichier"` pour obtenir le nombre seul, sans le nom du fichier.
- `[ -e "$element" ] || continue` : si le répertoire est vide, le motif `*` n'est pas remplacé et la boucle

tournerait une fois sur la chaîne littérale `rep/*`. Ce test l'évite.

- Les guillemets autour de `"$1"` et `"$element"` sont indispensables : sans eux, un nom contenant une espace serait découpé en plusieurs mots.

Exemple d'exécution :

```
$ ./ex4.sh ../tp3
Fichiers ordinaires : 9
Sous-répertoires   : 0
Taille totale      : 104547 octets

$ ./ex4.sh /nimportequoi
Erreur : le repertoire "/nimportequoi" n'existe pas
$ echo $?
1
```

Remarque : le motif `*` ignore les fichiers cachés (commençant par un point). C'est le comportement attendu ici ; pour les inclure il faudrait ajouter une seconde boucle sur `"$1"/.*`.